AUDITORIA DE SEGURANÇA APLICADA

Relatório de Auditoria de Segurança — Looply

Gestão operacional de conteúdo para agências de marketing/social (Demo V1)

Data: 28 de agosto de 2026

Versão do código: looply@750d2df (main) · verbumlab@311dd8c

Auditor: agente automatizado (opencode) com verificação manual arquivo por arquivo

Total de achados: 5 — 0 crítica · 2 alta · 0 média · 2 baixa · 1 informativa

Escopo auditado

Backend: /srv/verbum/workspaces/looply/backend — 24 arquivos de rotas, 15 services, 5 middlewares, lib (app.ts, env.ts, session.ts, password.ts, rateLimit.ts, loginLimiter.ts, validation.ts)

Frontend: /srv/verbum/workspaces/looply/frontend/src — componentes, features (inbox, contents, team, settings, media), lib/api.ts

Deploy/ops: ecosystem.config.cjs (pm2), ops/ (backup, nginx, systemd, deploy), .env.example, .gitignore

Histórico git (looply e verbumlab) e bundle de produção do frontend (apps/looply/frontend/assets)

Nota metodológica — mapeamento das categorias para a stack detectada

Stack detectada: linguagem: TypeScript (Node.js 24 / backend CommonJS; frontend ESM) · framework: Express 4 (backend) · React 19 + Vite (frontend SPA) · orm: Prisma 5 +

PostgreSQL 16 (sem RLS — isolamento por filtro manual de companyId) · auth: Sessão própria em cookie httpOnly (token aleatório 256-bit, sha256 em restingação), membership OWNER/ADMIN/MEMBER por empresa · frontend: React 19 + react-router + Tailwind; comunicação via cookie de sessão (sem tokens no bundle) · deploy: pm2 (ecosystem.config.cjs) + nginx proxy; sem Docker/Helm/Terraform no repositório.

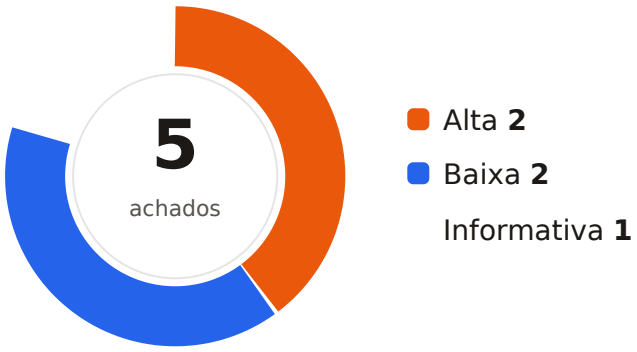
Cat. 1 (banco sem tranco): sem Supabase/RLS — o mecanismo de isolamento é o middleware de sessão (requireAuth) que resolve `req.auth.companyId` a partir da sessão (nunca do cliente) + filtro manual de companyId em cada service Prisma. Auditei todas as rotas de listagem/agregação/relatório e os where de cada service.
Cat. 2 (permissão no navegador): cruzei cada gate de papel do React (isOwner/isAdminOrOwner em Team, EditCompanyModal) com o endpoint correspondente (requireRole + checagens em service).
Cat. 3 (IDOR): percorri os 24 arquivos de rotas e os 15 services, handler por handler, verificando lookup por ID sempre pareado com companyId.
Cat. 4 (chaves expostas): grep de código/configs/CI/ops, .env.example, histórico git de ambos os repositórios (git grep em 200 commits + git log de .env) e bundle de produção do frontend.
Cat. 5 (XSS): frontend — dangerouslySetInnerHTML/eval/href dinâmicos/markdown; backend — templates de e-mail. Sem lib de sanitização no projeto (confirmado em ambos package.json).

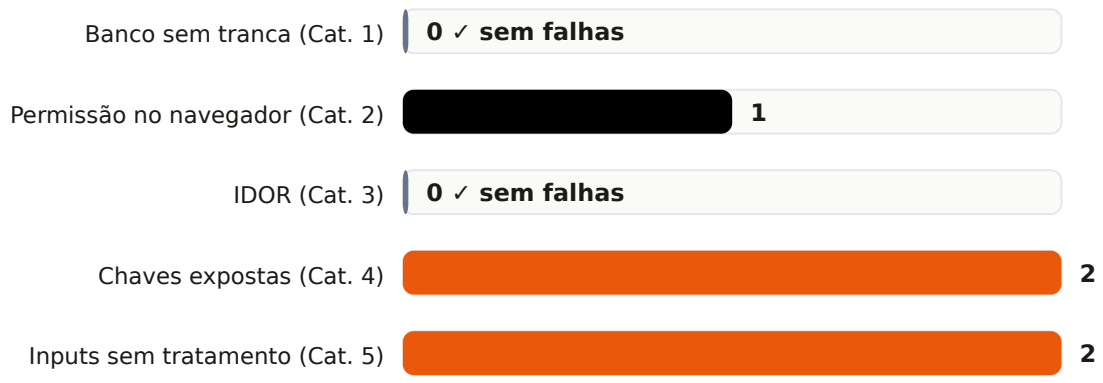
Resumo executivo

2 Alta

2 Baixa

O Looply tem uma base de autorização notavelmente consistente para um Demo V1: **zero falhas de isolamento de tenant e zero IDOR** nas 24 rotas auditadas — o companyId derivado da sessão acompanha toda query, incluindo agregações do dashboard e download de mídia por chave. Os riscos centrais estão fora desse núcleo: **um XSS armazenado** via validação permissiva de URL em mídias de conteúdo, e **a senha de administrador de produção commitada** no repositório vizinho verbumlab (script de captura). Os demais achados são de hardening.





Pontos fortes (verificado e correto)

Isolamento de tenant em 100% das rotas de listagem/agregação

Toda rota derivava req.auth.companyId do middleware (middleware/requireAuth.ts:17) e todo service filtra por companyId: dashboardService.ts:27-56 (groupBy), clientService.ts:63,111,131,137,145, jobService.ts:120,179,199,219,233,250,276, contentService.ts:114-241, appointmentService.ts:115, activityService.ts:29, approvalService.ts:33,76,87,96,100, attachmentService.ts:43,75,89,95,107,115,119, conversationService.ts:341,360,439,588,611. Nenhum handler de listagem/busca/relatório sem filtro de tenant.

Zero IDOR nos handlers por-ID

Todos os lookups por ID usam findFirst({ id, companyId }) antes de mutar (padrão verify-then-update). O endpoint por-chave /api/media/file valida prefixo do storageKey contra membership ativo (identityMediaService.ts:347-374). Arquivos de aprovação pública escopam por token+empresa+job (approvalService.ts:175-182).

Privilégio sempre revalidado no servidor

requireRole(OWNER, ADMIN) em settings.ts:53,86 e workflows.ts:38,54,74,91; checagem de papel no service em memberService.ts:38 (só OWNER gerencia papéis), :210 (ADMIN não reseta OWNER/ADMIN), :72 (convite), whatsappActorService.ts:71, media.ts:36-41 (requireAdminOrOwner). O frontend nunca é a única barreira.

Webhooks fail-closed

Meta: HMAC sha256 + timingSafeEqual com raw body (whatsappProvider.ts:100-111; rota whatsappWebhook.ts:76-86 — 503 se não configurado, 401 se assinatura inválida). Gupshup: allowlist de IP obrigatória, 503 sem configuração (whatsappGupshupWebhook.ts:52-63).

Segredos fora do repo do Looply

.env no .gitignore e nunca commitado (git log --all vazio para .env); .env.example só com placeholders CHANGE_ME; scripts ops/ exigem variáveis com :?required (ops/backup/verify.sh:32-33); bundle do frontend sem chaves (scan por sk-/AIza/ghp_/xox-); nenhuma credencial hardcode nos últimos 200 commits (git grep em ts/tsx/md/json).

Sessão e recuperação de senha robustas

Token de sessão hasheado (sha256) em banco (authService.ts:116); membership revalidado a cada request (:123-131); troca de senha revoga todas as sessões (auth.ts:107-118); recovery com token 256-bit hasheado, single-use, 1h de TTL, rate-limit e resposta que não vaza existência de conta (recoveryService.ts:27-67); rate-limit em login (10/15min), invitation-accept, password-reset e aprovações públicas (lib/rateLimit.ts).

Frontend sem sinks de XSS direto

Nenhum dangerouslySetInnerHTML/eval/new Function no src (grep). Links de chat aceitam só http/https (formatMessageText.tsx:40-42,62); DocumentBubble restringe href a http/https//api (DocumentBubble.tsx:27-33); avatares/logos vão apenas para <img src> (esquemas javascript:/data: são inertes em img).

Pontos fracos (riscos centrais)

1. Cadeia de confiança de mídia: a validação de URL aceita qualquer esquema e o frontend renderiza em href clicável (A1) — única rota de XSS encontrada.

2. Higiene de segredos entre repositórios: a credencial admin real vazou para o repo de marketing/captura (A2) — o repo do Looply em si está limpo.

3. Defesa de configuração: startup não rejeita placeholders de segredo (A4); e-mail usa input sem escape (A3).

Achados detalhados por categoria

SEVERIDADE	ACHADO (ARQUIVO:LINHA · CÓDIGO · EXPLORABILIDADE)

ALTA
A1 · XSS (Cat. 5)
	XSS armazenado via mediaUrl com scheme javascript:
	`backend/src/lib/validation.ts:18,25 · frontend/src/features/ContentDetail.tsx:889`
	`// validation.ts:25 — z.string().url() aceita "javascript:alert(1)" (verificado empiricamente)`
	`mediaUrl: z.string().url().max(2000).optional().or(z.literal("")).optional(),`
	`// ContentDetail.tsx:889 — href recebe a URL sem restrição de esquema`
	`<a href={v.mediaUrl} target="_blank" rel="noopener noreferrer">Ver mídia</a>`
	Membro autenticado do workspace cria uma versão de conteúdo (POST /api/contents/:id/versions) com mediaUrl="javascript:payload". O validador zod .url() aceita qualquer esquema com dois-pontos (testado: javascript: e data: passam). Outro usuário da mesma equipe abre o conteúdo e clica em "Ver mídia" — o anchor executa o scheme javascript: no contexto da aplicação. Não há biblioteca de sanitização no projeto. Condições: atacante precisa ser membro convidado + vítima clica; reflete-se em roubo de cookie de sessão (httpOnly mitiga exfiltração direta, mas o script roda como a vítima na origem).

ALTA
A2 · Chaves expostas (Cat. 4)
	Senha do admin de produção commitada no repositório verbumlab
	`/srv/verbum/workspaces/verbumlab/scripts/capture-looply.ts:32`
	`await page.fill('input[type="password"]', 'B7_PiXWC+Y4mQ*GG4k@7YWX_u02BU^Qq');`
	É a SEED_ADMIN_PASSWORD real da instância pública https://looply.verbumlab.com.br — validada em execução: o login com essa senha retornou 200 e sessão ativa. Está commitada em dois commits do histórico do verbumlab (47b42b8 e 0816379). Qualquer leitor do repositório obtém acesso de administrador (persona OWNER) à instância de produção do Looply, incluindo dados de todos os workspaces de demonstração e rotas administrativas.

BAIXA
A3 · Inputs sem tratamento (Cat. 5)
	Nome do usuário sem escape no HTML do e-mail de recuperação
	`backend/src/services/mailService.ts:79 (fonte do dado: routes/auth.ts:169-186)`
	`Olá, <strong style="color:#2A171F;">${input.name}</strong>!  // mailService.ts:79`
	`// name é livre: updateProfileSchema aceita qualquer string de até 120 chars (auth.ts)`
	PATCH /api/auth/me permite gravar name com marcação HTML. O template do e-mail de recuperação injeta ${input.name} sem escape. O e-mail só é enviado para o próprio endereço do usuário (self-injection), limitando o impacto a phishing/injeção visual no cliente de e-mail do próprio atacante — por isso baixa. Ainda é um sink real de HTML não sanitizado no backend.

SEVERIDADE	ACHADO (ARQUIVO:LINHA · CÓDIGO · EXPLORABILIDADE)

BAIXA
A4 · Chaves expostas (Cat. 4)
	validateEnv não rejeita valores placeholder/default de segredo
	`backend/src/lib/env.ts:68-85`
	`if (!process.env.DATABASE_URL) missing.push("DATABASE_URL");`
	`if (env.CORS_ORIGIN === "*") throw new Error("CORS_ORIGIN must not be wildcard");`
	`// única validação: presença de DATABASE_URL e CORS não-curinga`
	A validação de startup checa presença, não valor. SEED_ADMIN_PASSWORD="CHANGE_ME_SEED_ADMIN_PASSWORD" (o placeholder do .env.example) passa direto e vira a senha de admin se ninguém sobrescrever — exatamente o padrão "default público que vira segredo real". Os segredos de webhook (WHATSAPP_APP_SECRET etc.) falham fechado quando ausentes (bom), mas valores placeholder óbvios não são rejeitados em nenhum ponto.

INFORMATIVA
A5 · Permissão no navegador (Cat. 2)
	Gate de papel do frontend trata role ausente como privilegiado (fail-open na UI)
	`frontend/src/features/Team.tsx:62-63 · frontend/src/features/EditCompanyModal.tsx:48`
	`const isOwner = currentRole === "OWNER" || !currentRole; // Team.tsx:62`
	`const isAdminOrOwner = currentRole === "OWNER" || currentRole === "ADMIN" || !currentRole; // :63`
	`const isOwnerOrAdmin = activeRole === "OWNER" || activeRole === "ADMIN" || !activeRole; // EditCompanyModal.tsx:48`
	Com role indefinida (edge: membership ainda carregando), a UI exibe controles de OWNER/ADMIN. O servidor valida em toda rota sensível (requireRole + checagens de service — verificado handler por handler), então o pior caso é um 403; não há bypass real. Registrado como informativa/hardening de UX.

Recomendações priorizadas

P1 **Rotacionar a senha admin do Looply e purgar a credencial do repositório verbumlab** (A2) — trocar SEED_ADMIN_PASSWORD no .env, remover do script de captura (ler de env), reescrever/invalidar os 2 commits com a senha e considerar o histórico exposto.

P2 **Restringir esquema de URL em mediaUrl/logoUrl/avatarUrl** (A1) — substituir z.string().url() por refine que aceite apenas https:// (e, para mídia, o domínio de storage); sanitizar a renderização ou abrir links de mídia só após validação. Adiciona defesa em profundidade: CSP via helmet (script-src 'self').

P3 **Escapar input.name no template de e-mail** (A3) — função de escape HTML mínima ou renderizar em texto puro;idealmente mover nome para fora de marcação.

P4 **validateEnv rejeita placeholders** (A4) — deny-list de valores (CHANGE_ME, password, secret…) para SEED_ADMIN_PASSWORD e segredos de webhook em produção (NODE_ENV=production ⇒ falha de startup).

P5 **Gate de papel fail-closed na UI** (A5) — tratar role ausente como MEMBER (esconder controles) em Team.tsx e EditCompanyModal.tsx; mantém o backend como está (já correto).

Issues para o GitHub

Copie e cole cada bloco como uma issue. Achados triviais relacionados foram mantidos individuais por tocarem repositórios/times diferentes.

--- ISSUE 1 ---

````
**[Segurança] XSS armazenado via mediaUrl com scheme javascript:**

Labels: `security, alta`

## Descrição do problema
Membro autenticado do workspace cria uma versão de conteúdo (POST
/api/contents/:id/versions) com mediaUrl="javascript:payload". O validador zod .url()
aceita qualquer esquema com dois-pontos (testado: javascript: e data: passam). Outro
usuário da mesma equipe abre o conteúdo e clica em "Ver mídia" — o anchor executa o
scheme javascript: no contexto da aplicação. Não há biblioteca de sanitização no projeto.
Condições: atacante precisa ser membro convidado + vítima clica; reflete-se em roubo de
cookie de sessão (httpOnly mitiga exfiltração direta, mas o script roda como a vítima na
origem).

## Evidência
`backend/src/lib/validation.ts:18,25 · frontend/src/features/ContentDetail.tsx:889`

```
// validation.ts:25 — z.string().url() aceita "javascript:alert(1)" (verificado
empiricamente)
mediaUrl: z.string().url().max(2000).optional().or(z.literal("")).optional(),
// ContentDetail.tsx:889 — href recebe a URL sem restrição de esquema
Ver mídia
```

## Impacto
Comprometimento de conta/sessão de usuário autenticado ou de administrador da instância.

## Sugestão de correção
Trocar `z.string().url()` por `z.string().refine(u => /^https:\/\//.test(u))` nos schemas
de mediaUrl/logoUrl/avatarUrl (validation.ts, auth.ts, settings.ts) e adicionar CSP
`script-src 'self'` no helmet.

## Critérios de aceite
- [ ] POST /api/contents/:id/versions com mediaUrl="javascript:alert(1)" retorna 400
- [ ] Conteúdo existente com mediaUrl javascript: renderiza link inerte/bloqueado
- [ ] Teste unitário cobrindo schemes não-https rejeitados
- [ ] helmet CSP ativo em produção
````

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

````
**[Segurança] Senha do admin de produção commitada no repositório verbumlab**

Labels: `security, alta`

## Descrição do problema
É a SEED_ADMIN_PASSWORD real da instância pública https://looply.verbumlab.com.br —
validada em execução: o login com essa senha retornou 200 e sessão ativa. Está commitada
em dois commits do histórico do verbumlab (47b42b8 e 0816379). Qualquer leitor do
repositório obtém acesso de administrador (persona OWNER) à instância de produção do
Looply, incluindo dados de todos os workspaces de demonstração e rotas administrativas.

## Evidência
`/srv/verbum/workspaces/verbumlab/scripts/capture-looply.ts:32`

```
await page.fill('input[type="password"]', 'B7_PiXWC+Y4mQ*GG4k@7YWX_u02BU^Qq');
```

## Impacto
Comprometimento de conta/sessão de usuário autenticado ou de administrador da instância.

## Sugestão de correção
Rotacionar a senha, ler credenciais do script via process.env (não commitar), invalidar
os commits antigos (filter-repo/BFG) e auditar access logs de login admin.

## Critérios de aceite
- [ ] Senha antiga não autentica mais (verificado em /login)
- [ ] capture-looply.ts lê credenciais de process.env
- [ ] git log -S"senha antiga" não retorna resultado no verbumlab
- [ ] .env atualizado com nova senha fora do repo
````

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

````
**[Segurança] Nome do usuário sem escape no HTML do e-mail de recuperação**

Labels: `security, baixa`

## Descrição do problema
PATCH /api/auth/me permite gravar name com marcação HTML. O template do e-mail de
recuperação injeta ${input.name} sem escape. O e-mail só é enviado para o próprio
endereço do usuário (self-injection), limitando o impacto a phishing/injeção visual no
cliente de e-mail do próprio atacante — por isso baixa. Ainda é um sink real de HTML não
sanitizado no backend.

## Evidência
`backend/src/services/mailService.ts:79 (fonte do dado: routes/auth.ts:169-186)`

```
Olá, ${input.name}!  // mailService.ts:79
// name é livre: updateProfileSchema aceita qualquer string de até 120 chars (auth.ts)
```

## Impacto
Injeção limitada / risco de configuração insegura silenciosa.

## Sugestão de correção
Aplicar escape HTML (função de 5 linhas) em `input.name` antes de interpolar no template
de mailService.ts.

## Critérios de aceite
- [ ] E-mail de recuperação com name contendo  exibe o texto literal, não marcação
- [ ] Teste unitário do template com nome hostil
````

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

````
**[Segurança] validateEnv não rejeita valores placeholder/default de segredo**

Labels: `security, baixa`

## Descrição do problema
A validação de startup checa presença, não valor.
SEED_ADMIN_PASSWORD="CHANGE_ME_SEED_ADMIN_PASSWORD" (o placeholder do .env.example) passa
direto e vira a senha de admin se ninguém sobrescrever — exatamente o padrão "default
público que vira segredo real". Os segredos de webhook (WHATSAPP_APP_SECRET etc.) falham
fechado quando ausentes (bom), mas valores placeholder óbvios não são rejeitados em
nenhum ponto.

## Evidência
`backend/src/lib/env.ts:68-85`

```
if (!process.env.DATABASE_URL) missing.push("DATABASE_URL");
if (env.CORS_ORIGIN === "*") throw new Error("CORS_ORIGIN must not be wildcard");
// única validação: presença de DATABASE_URL e CORS não-curinga
```

## Impacto
Injeção limitada / risco de configuração insegura silenciosa.

## Sugestão de correção
Em validateEnv(), quando NODE_ENV=production, rejeitar valores da deny-list (CHANGE_ME,
exemplo, password123…) para SEED_ADMIN_PASSWORD e segredos de webhook.

## Critérios de aceite
- [ ] NODE_ENV=production + SEED_ADMIN_PASSWORD=CHANGE_ME_SEED_ADMIN_PASSWORD aborta o
startup
- [ ] Valores legítimos passam normalmente (teste de regressão)
````

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

````
**[Hardening] Gate de papel do frontend trata role ausente como privilegiado (fail-open
na UI)**

Labels: `security, informativa`

## Descrição do problema
Com role indefinida (edge: membership ainda carregando), a UI exibe controles de
OWNER/ADMIN. O servidor valida em toda rota sensível (requireRole + checagens de service
— verificado handler por handler), então o pior caso é um 403; não há bypass real.
Registrado como informativa/hardening de UX.

## Evidência
`frontend/src/features/Team.tsx:62-63 · frontend/src/features/EditCompanyModal.tsx:48`

```
const isOwner = currentRole === "OWNER" || !currentRole;            // Team.tsx:62
const isAdminOrOwner = currentRole === "OWNER" || currentRole === "ADMIN" ||
!currentRole; // :63
const isOwnerOrAdmin = activeRole === "OWNER" || activeRole === "ADMIN" || !activeRole;
// EditCompanyModal.tsx:48
```

## Impacto
Somente cosmético — servidor já valida; melhoria de UX de segurança.

## Sugestão de correção
Nos componentes Team/EditCompanyModal, papel ausente deve resolver para o mínimo
privilégio (MEMBER), não para OWNER/ADMIN.

## Critérios de aceite
- [ ] Team.tsx com currentRole undefined não exibe controles de gestão
- [ ] EditCompanyModal com activeRole undefined não exibe campos administrativos
````

--- FIM ISSUE 5 ---